

1260. Shift 2D Grid

Given a 2D grid of size $m \times n$ and an integer k . You need to shift the grid k times.

In one shift operation:

- Element at $\text{grid}[i][j]$ moves to $\text{grid}[i][j + 1]$.
- Element at $\text{grid}[i][n - 1]$ moves to $\text{grid}[i + 1][0]$.
- Element at $\text{grid}[m - 1][n - 1]$ moves to $\text{grid}[0][0]$.

Return the 2D grid after applying shift operation k times.

Example 1:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \rightarrow \begin{bmatrix} 9 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$$

Input: $\text{grid} = [[1,2,3],[4,5,6],[7,8,9]]$, $k = 1$

Output: $[[9,1,2],[3,4,5],[6,7,8]]$

Example 2:

$$\begin{bmatrix} 3 & 8 & 1 & 9 \\ 19 & 7 & 2 & 5 \\ 4 & 6 & 11 & 10 \\ 12 & 0 & 21 & 13 \end{bmatrix} \rightarrow \begin{bmatrix} 13 & 3 & 8 & 1 \\ 9 & 19 & 7 & 2 \\ 5 & 4 & 6 & 11 \\ 10 & 12 & 0 & 21 \end{bmatrix} \rightarrow \begin{bmatrix} 21 & 13 & 3 & 8 \\ 1 & 9 & 19 & 7 \\ 2 & 5 & 4 & 6 \\ 11 & 10 & 12 & 0 \end{bmatrix}$$
$$\rightarrow \begin{bmatrix} 0 & 21 & 13 & 3 \\ 8 & 1 & 9 & 19 \\ 7 & 2 & 5 & 4 \\ 6 & 11 & 10 & 12 \end{bmatrix} \rightarrow \begin{bmatrix} 12 & 0 & 21 & 13 \\ 3 & 8 & 1 & 9 \\ 19 & 7 & 2 & 5 \\ 4 & 6 & 11 & 10 \end{bmatrix}$$

Input: grid = [[3,8,1,9],[19,7,2,5],[4,6,11,10],[12,0,21,13]], k = 4

Output: [[12,0,21,13],[3,8,1,9],[19,7,2,5],[4,6,11,10]]

Example 3:

Input: grid = [[1,2,3],[4,5,6],[7,8,9]], k = 9

Output: [[1,2,3],[4,5,6],[7,8,9]]

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m <= 50
- 1 <= n <= 50
- -1000 <= grid[i][j] <= 1000
- 0 <= k <= 100

In-place Rotation | 2D-1D Index Mapping | Beats 100%

```
class Solution {
public:
    vector<vector<int>> shiftGrid(vector<vector<int>>& grid, int k) {
        if (!k) return grid;
        int r = grid.size(), c = grid[0].size();
        int n = r * c;

        k = k % n;
        if (!k) return grid;

        auto shift = [&](int i, int j) {
            while (i < j) {
```

```

        swap(grid[i / c][i % c], grid[j / c][j % c]);

        i++;

        j--;

    }

};

shift(0, n - 1);

shift(0, k - 1);

shift(k, n - 1);

return grid;

}

};

```

Example:

- Shift the grid by 3:

Grid

0	1	2	3
4	5	6	7
8	9	10	11

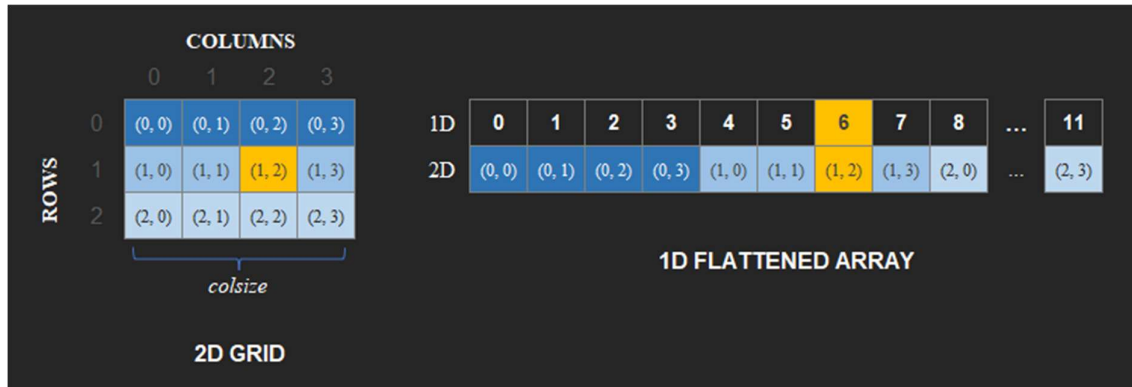
Virtual 1D

<i>val</i>	0	1	2	3	4	5	6	7	8	9	10	11
1D	0	1	2	3	4	5	6	7	8	9	10	11
2D	(0, 0)	(0, 1)	(0, 2)	(0, 3)	(1, 0)	(1, 1)	(1, 2)	(1, 3)	(2, 0)	(2, 1)	(2, 2)	(2, 3)

The values are equal to the 1D index for simplification.

Index Mapping

Let's recall how the 2D grid coordinates virtually maps into a 1D flattened array.



To find the linear index from grid coordinate (row, col), and colsize:

$$\text{index}(1D) = (\text{row} * \text{colsize}) + \text{col}$$

In-place Shifting

Requires [189. Rotate Array](#) Intuition.

1.) Reverse

1D	11	10	9	8	7	6	5	4	3	2	1	0
2D	(2, 3)	(2, 2)	(2, 1)	(2, 0)	(1, 3)	(1, 2)	(1, 1)	(1, 0)	(0, 3)	(0, 2)	(0, 1)	(0, 0)
	0	1	2	3	4	5	6	7	8	9	10	11

2.) Reverse (0, k-1)

1D	9	10	11	8	7	6	5	4	3	2	1	0
2D	(2, 1)	(2, 2)	(2, 3)	(2, 0)	(1, 3)	(1, 2)	(1, 1)	(1, 0)	(0, 3)	(0, 2)	(0, 1)	(0, 0)
	0	1	2	3	4	5	6	7	8	9	10	11

k

3.) Reverse (k, n-1)

1D	9	10	11	0	1	2	3	4	5	6	7	8
2D	(2, 1)	(2, 2)	(2, 3)	(0, 0)	(0, 1)	(0, 2)	(0, 3)	(1, 0)	(1, 1)	(1, 2)	(1, 3)	(2, 0)
	0	1	2	3	4	5	6	7	8	9	10	11

k

The **shift(l, j)** function takes in **linear index i** and **j**, then we swap their values.

The indexes are remapped to 2D grid.

Remapping 1D to 2D

To find the (row, col) from a linear index:

row = [index(1D) / colsize]

col = index(1D) % colsize

Return the **shifted grid**:

Shifted Grid			
9	10	11	0
1	2	3	4
5	6	7	8

Time Complexity: O(n)

Space Complexity: O(1)

JAVA:

```
class Solution {  
    private void reverse(int start, int end, int[] arr) {  
        while (start < end) {  
            int t = arr[start];  
            arr[start] = arr[end];  
            arr[end] = t;  
            start++; end--;  
        }  
    }  
  
    public List<List<Integer>> shiftGrid(int[][] grid, int k) {  
        int r = grid.length;  
        int c = grid[0].length;  
        int n = r * c;  
        int index = 0;  
        k = k % n;  
  
        List<List<Integer>> list = new ArrayList<>();  
        int[] arr = new int[n];  
  
        for (int i = 0; i < r; ++i) {  
            for (int j = 0; j < c; ++j) {  
                arr[index++] = grid[i][j];  
            }  
        }  
  
        reverse(0, n - 1, arr);  
        reverse(0, k - 1, arr);  
    }  
}
```

```
reverse(k, n - 1, arr);
```

```
index = 0;
```

```
for (int i = 0; i < r; ++i) {
```

```
    List<Integer> row = new ArrayList<>();
```

```
    for (int j = 0; j < c; ++j) {
```

```
        row.add(arr[index++]);
```

```
    }
```

```
    list.add(row);
```

```
}
```

```
return list;
```

```
}
```

```
}
```